Nosh_Robotics_Technical_Report_Draft

Egg Segmentation and Object Tracking

Technical Report | Computer Vision Assignment

Submitted by: Dishank Patel

Institution: D. Y. Patil College of Engineering, Akurdi

Department: Robotics and Automation

Organization: Nosh Robotics

Assignment: Egg Segmentation and Individual Object Tracking

Date: 01/11/2026

---

1. Abstract

This project explores computer vision techniques for egg segmentation and individual object tracking in video sequences.

The primary objective was to develop and evaluate an image-processing pipeline capable of identifying egg regions, generating segmentation masks, evaluating segmentation performance, and implementing an object-tracking approach.

The work involved two segmentation approaches: Segment Anything Model (SAM) and classical computer vision techniques. Multiple experiments were conducted to improve classical segmentation using image preprocessing, colour-space transformations, thresholding, contour analysis, morphological operations, and GrabCut refinement.

The classical segmentation experiments were evaluated against SAM-generated pseudo-label masks using Intersection over Union (IoU) and Dice coefficient.

A Kalman Filter and Hungarian assignment-based tracking implementation was also developed. The implementation generates annotated tracking video and CSV tracking data. Tracking identity consistency and performance require further validation before quantitative conclusions can be made.

The experiments demonstrate the importance of preprocessing, parameter selection, evaluation methodology, and failure-case analysis when developing practical computer vision systems.

2. Introduction

Object segmentation and tracking are important components of computer vision systems used in industrial automation, robotics, inspection, and object-handling applications.

The objective of this assignment was to investigate egg segmentation and tracking using computer vision techniques.

The project was divided into the following major tasks:

1. Dataset inspection and preparation.
2. Reference-mask generation using SAM.
3. Classical image segmentation.
4. Experimentation and parameter optimization.
5. Quantitative segmentation evaluation.
6. Failure-case inspection.
7. Implementation of individual object tracking.

The development process focused on building a reproducible workflow and comparing different approaches rather than relying on a single segmentation technique.

3. Objectives

The major objectives were:

- Understand the dataset and image characteristics.
- Generate reference segmentation masks using SAM.
- Implement classical computer vision-based segmentation.
- Experiment with different preprocessing and segmentation techniques.
- Evaluate segmentation using IoU and Dice coefficient.
- Identify failure cases and limitations.
- Implement object tracking using Kalman filtering and Hungarian assignment.
- Maintain reproducible code and experiment documentation.

4. Tools and Technologies

Component	Technology
Programming language	Python
Image processing	OpenCV
Numerical operations	NumPy
Data analysis	Pandas
Reference segmentation	Segment Anything Model (SAM)
Segmentation refinement	GrabCut

Component	Technology
Object tracking	Kalman Filter
Data association	Hungarian Algorithm
Testing	Python unittest
Development environment	Ubuntu Linux

Environment versions recorded during development:

- Python: 3.10.12
- OpenCV: 5.0.0
- NumPy: 2.2.6

SAM model: ViT-B checkpoint.

5. Dataset and Preparation

The dataset contained 110 images.

For experimentation, the images were organized into two custom splits:

Split	Number of Images
Development	88
Evaluation	22
Total	110

The two splits were disjoint and together covered all 110 images.

The dataset was inspected to understand image dimensions, object appearance, illumination variations, background conditions, and segmentation challenges.

5.1 Dataset Challenges

The main challenges observed during segmentation included:

- Variations in egg orientation.
- Changes in illumination.
- Low contrast between eggs and the background.
- Irregular object boundaries.
- Background regions with colours similar to the eggs.
- Overlapping or closely positioned objects.
- Sensitivity of thresholding methods to image conditions.

These challenges influenced the preprocessing and segmentation experiments.

6. Reference Segmentation Using SAM

The Segment Anything Model (SAM) was used to generate reference segmentation masks.

The ViT-B model checkpoint was used to process the dataset and generate masks for the images.

The generated masks were used as pseudo-labels for comparing the classical segmentation approaches.

6.1 Purpose of SAM

SAM provided a consistent reference for evaluating the segmentation experiments.

The reference masks were checked for:

- Image correspondence.
- Mask dimensions.
- File readability.
- Availability of generated masks.

All 110 images had corresponding readable, dimension-matched SAM masks.

6.2 Important Evaluation Consideration

The SAM masks are pseudo-labels rather than independently verified ground-truth annotations.

Therefore, the reported IoU and Dice values measure agreement between the classical segmentation output and SAM-generated masks.

They should not be interpreted as independently verified segmentation accuracy.

7. Classical Segmentation Approach

A classical computer vision pipeline was developed to segment egg regions without relying on a deep-learning segmentation model during inference.

The experiments focused on image preprocessing, thresholding, contour selection, and GrabCut refinement.

7.1 Selected Approach: Trial 5

Trial 5 was selected as the final classical segmentation baseline because it provided the strongest overall balance among the tested experiments.

The pipeline consisted of the following operations:

Step 1: Colour-space transformation

The input image was converted into the LAB colour space.

The L channel was used for illumination-related processing.

Step 2: Illumination correction

Gaussian background estimation with a sigma value of 80 was applied to reduce the effect of illumination variation.

Step 3: Image normalization

The illumination-corrected image was normalized to improve consistency.

Step 4: Noise reduction

A median filter with a 5 × 5 kernel was applied.

Step 5: Thresholding

Otsu thresholding was applied in both normal and inverted polarities to generate candidate foreground masks.

Step 6: Morphological processing

Morphological operations using a 5 × 5 kernel were applied to improve mask continuity and reduce small unwanted regions.

Step 7: Contour selection

Contours were filtered using geometric conditions:

- Area between 2% and 40% of the image area.
- Aspect ratio not greater than 2.8.
- Circularity greater than or equal to 0.25.

Step 8: Fallback segmentation

A central elliptical region was used to initialize GrabCut when the primary segmentation approach did not produce a suitable candidate.

8. Experimental Trials

Multiple experiments were conducted to investigate the effect of different segmentation strategies.

Trial	Approach	Mean IoU	Mean Dice	Zero-IoU Cases
Trial 5	Baseline segmentation	0.4029	0.4650	30
Trial 6	Ellipse refinement	0.4023	0.4663	31
Trial 7	LAB-guided GrabCut	0.3024	0.3404	45
Trial 8	Conservative refinement	0.4033	0.4653	30
Trial 9	Multi-candidate segmentation	0.2117	0.2956	14
Trial 10	Conservative candidate generation	0.2375	0.3112	31

Note: Zero-IoU cases represent images where the evaluated segmentation had zero overlap with the corresponding SAM pseudo-label.

8.1 Experimental Observations

Trial 5 — Baseline

Trial 5 provided a balanced baseline across the evaluated experiments.

Trial 6 — Ellipse Refinement

The results were nearly identical to Trial 5, with a small increase in mean Dice but a slight reduction in mean IoU.

Trial 7 — LAB-Guided GrabCut

This experiment resulted in lower mean IoU and Dice scores and a higher number of zero-IoU cases.

Trial 8 — Conservative Refinement

The results were very close to Trial 5. The small improvement in mean IoU was not sufficient to establish a meaningful overall improvement.

Trial 9 — Multi-Candidate Segmentation

This approach produced substantially lower mean IoU and Dice scores.

Trial 10 — Conservative Candidate Generation

This experiment also performed below the selected baseline.

8.2 Selected Experiment

Trial 5 was retained as the final classical segmentation baseline.

The later experiments did not demonstrate a sufficiently consistent improvement to justify replacing it.

9. Quantitative Evaluation

The segmentation pipeline was evaluated using Intersection over Union and Dice coefficient.

9.1 Intersection over Union (IoU)

IoU measures the overlap between the predicted segmentation mask and the reference mask.

[

IoU = \frac{|A \cap B|}{|A \cup B|}

]

Where:

- A represents the predicted segmentation mask.
- B represents the reference segmentation mask.

A higher IoU indicates greater overlap between the two masks.

9.2 Dice Coefficient

The Dice coefficient measures the similarity between two segmentation masks.

[

Dice = \frac{2|A \cap B|}{|A| + |B|}

]

A higher Dice score indicates greater similarity.

9.3 Development Results

Trial 5 was evaluated on 88 development images.

Metric	Result
Mean IoU	0.4029
Median IoU	0.4069
Mean Dice	0.4650
Median Dice	0.5784
Zero-IoU Cases	30 / 88

9.4 Evaluation Results

Trial 5 was also evaluated on 22 evaluation images.

Metric	Result
Mean IoU	0.3576
Median IoU	0.2469
Mean Dice	0.4253
Median Dice	0.3958
Zero-IoU Cases	8 / 22

9.5 Evaluation Interpretation

The evaluation results were lower than the development results.

This indicates that the selected classical segmentation pipeline did not generalize equally well across all images.

The difference between mean and median scores also suggests that segmentation performance varied considerably across individual images.

The zero-IoU cases highlight images where the pipeline failed to produce meaningful overlap with the reference mask.

The results indicate that further improvements are needed in object localization, foreground separation, and handling of difficult illumination and background conditions.

Evaluation limitation: The evaluation split was not established as a fully independent blind test set. The results should therefore be treated as an evaluation on the defined custom split, not as an unbiased estimate of performance on unseen real-world data.

10. Failure Analysis

Failure-case analysis was performed to understand why the segmentation pipeline did not perform consistently.

The analysis focused on zero-IoU images, difficult segmentation examples, and fallback behaviour.

10.1 Observed Challenges

The main areas requiring attention include:

- Incorrect foreground and background separation.
- Inaccurate object localization.
- Sensitivity to illumination.
- Irregular segmentation boundaries.
- Cases where thresholding did not isolate the egg region.
- Cases where the fallback initialization did not provide a suitable segmentation.

10.2 Importance of Failure Analysis

Failure analysis helps identify limitations that are not visible from aggregate metrics alone.

It also provides direction for future improvements, including:

- Better foreground candidate generation.
- More robust object localization.
- Improved handling of overlapping objects.
- Adaptive preprocessing.
- Better initialization for segmentation refinement.
- Evaluation using independently verified annotations.

11. Reproducibility and Testing

The final workflow was organized into reusable stages.

The implementation in `src/egg_workflow.py` contains the E0–E5 workflow.

The selected Trial 5 implementation was reproduced on the 88 development images.

The reproduction comparison achieved:

- 88 out of 88 images reproduced.
- Maximum IoU difference of approximately 0.0000005.

The small difference is consistent with rounding in the stored CSV metrics.

11.1 Unit Testing

Automated tests were implemented using Python's unittest framework.

The test suite contained six tests, and all six passed in the recorded test run.

Command used:

```
.venv/bin/python -m unittest discover -s tests -v
```

This helped verify the behaviour of the implemented workflow components.

12. Individual Object Tracking

An individual object-tracking implementation was developed using a combination of Kalman filtering and Hungarian assignment.

The implementation is available in:
`src/track_eggs_kalman_hungarian.py`

12.1 Kalman Filter

The Kalman Filter is used to estimate object states over time.

It predicts the expected object position in the next frame and updates the estimate when a new detection is available.

This helps maintain object trajectories when detections vary between frames.

12.2 Hungarian Assignment

The Hungarian algorithm is used for associating current-frame detections with existing tracks.

It provides an assignment strategy for matching detections to predicted track positions.

12.3 Tracking Outputs

The implementation generated the following local artifacts:

- `corrected_tracked_eggs.mp4`
- `corrected_tracks.csv`

The video contains the tracking visualization, while the CSV stores tracking information.

12.4 Tracking Evaluation Status

The implementation and output generation have been completed.

However, quantitative tracking performance and identity continuity across the complete video still require validation.

The following aspects should be inspected before reporting final tracking results:

- Whether object IDs remain consistent across frames.
- Whether IDs switch when objects overlap or move close to one another.
- Whether missed detections cause track loss.
- Whether new detections create duplicate tracks.
- Whether tracks are correctly initialized and terminated.

No numerical tracking accuracy or identity-switch rate is claimed at this stage.

13. Limitations

The current implementation has several limitations.

1. The segmentation pipeline is sensitive to variations in image conditions.
2. Classical thresholding and contour-based methods may fail when the foreground and background have similar appearance.
3. The reference masks are SAM-generated pseudo-labels rather than independently verified ground truth.
4. The evaluation split is not a fully independent blind test set.
5. Segmentation performance varies considerably between images.
6. Tracking identity consistency has not yet been quantitatively validated.
7. Further evaluation is required before making claims about real-world tracking reliability.

14. Future Improvements

Potential improvements include:

Segmentation

- Investigate adaptive thresholding and more robust foreground extraction.
- Improve object candidate localization.
- Explore alternative colour-space combinations.
- Improve handling of overlapping eggs.
- Evaluate against manually verified ground-truth masks.

Tracking

- Validate identity continuity over the full video.
- Investigate detection-to-track association costs.
- Tune Kalman Filter parameters.
- Handle missed detections and temporary occlusions.
- Evaluate using tracking-specific metrics such as ID switches and track fragmentation.

Evaluation

- Establish a fully independent evaluation set.
- Add manually verified segmentation annotations.
- Perform systematic failure-case analysis.
- Maintain reproducible experiment configurations.

15. Conclusion

This project involved the development and evaluation of egg segmentation and object-tracking approaches using computer vision.

SAM was used to generate reference pseudo-labels, while classical image-processing methods were investigated through multiple experimental trials.

Trial 5 was selected as the final classical segmentation baseline because it provided the most balanced results among the tested approaches.

The evaluation highlighted the limitations of traditional segmentation methods under varying image conditions and demonstrated the importance of quantitative evaluation and failure analysis.

A Kalman Filter and Hungarian assignment-based tracking implementation was also developed, producing annotated video and CSV outputs. Further validation is required to establish tracking reliability and identity consistency.

Overall, the project provided practical experience in image preprocessing, segmentation, evaluation, experimentation, reproducibility, and multi-object tracking.

16. Project Structure

The project was organized to separate source code, experiments, data, models, and generated outputs.

Important source files include:

```
nosh_cv_assignment/
│
├── src/
│   ├── egg_workflow.py
```

```
│   └── track_eggs_kalman_hungarian.py
│
├── models/
│   └── SAM checkpoint
│
├── splits/
│   ├── dev.txt
│   └── eval.txt
│
├── tests/
│   └── test_egg_workflow.py
│
├── archive/
│   └── experiments/
│
├── README.md
├── experiment_log.md
└── requirements.txt
```

Generated outputs, datasets, virtual environments, and large model checkpoints are maintained separately from the source-code submission.

17. Reproduction Instructions

Clone the repository and install the required dependencies.

```
git clone <PUBLIC_GITHUB_REPOSITORY_URL>
cd nosh_cv_assignment
```

Create and activate a virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Run the unit tests:

```
python -m unittest discover -s tests -v
```

Refer to the README and experiment log for additional implementation and experiment details.

18. AI Tools and Technical Guidance Disclosure

During the project, I referred to technical documentation, online resources, and external technical guidance to explore suitable approaches to computer vision problems.

The Kalman Filter and Hungarian Algorithm-based tracking approach was suggested by the Deep Learning Lead at my current organization. I used this technical direction to implement the tracking pipeline and generate tracking outputs.

The segmentation workflow, experimental trials, preprocessing techniques, parameter selection, evaluation, and failure analysis were carried out as part of my own development and experimentation.

AI-based tools were used selectively for understanding technical concepts, troubleshooting specific implementation issues, and improving documentation. All experiments were executed, and their outputs and results were reviewed by me.

These resources served as supplementary technical assistance throughout the project rather than replacing the hands-on development and experimentation.